

HAETAE-IRV

잔차 δ 는 어떻게 계산되는가

논문 알고리즘 2 — 라인별 설명 · 실제 값 예시 · 구현 코드

$\delta = \text{Chk_M1} \mid \text{Chk_M2} \mid \text{Chk_M3} \mid \text{Chk_M4} \mid \text{Chk_M5} \mid \text{Chk_M6}$

CLEAR THIS UP

핵심 — “슈도코드를 라인별로 다시 실행”이 아니다

오해 $\times$

“원본 Sign의 1~N행을 통째로
한 번 더 실행해서 비교한다”

= 전체 이중연산(double)

→ **비용 2.0x**

IRV 실제 $\checkmark$

표적별 검사 6개(M1~M6)를 한 번씩
수행 → 잔차를 δ 에 OR로 누적

= 응답연산만 1회 재계산 + 가벼운 재검사

→ **비용 1.14x**

검사 중 하나라도 잔차 $\neq 0 \rightarrow \delta \neq 0 \rightarrow$ 서명 전체 난수화. “모든 검사 통과($\delta=0$)”만 서명이 살아남는다.

WHY VULNERABLE

(복습) 오류 지점이 왜 취약한가

비밀키를 쓰는 유일한 연산: $z = y + (-1)^b \cdot LN \cdot (c \cdot s1)$ (y=마스크, c=공개, s1=비밀)

지점	오류	방출 z	왜 위험한가
T2	+y 스킵	$z = LN \cdot c \cdot s1$	마스크 제거 → 서명 1개로 s1 100% 복원
T1	c·s 스킵	$z \approx y$	정상과 차분 → $LN \cdot c \cdot s1 \rightarrow s1$ 복원
RB	거부 스킵	경계초과 z	B1 넘긴 z가 통계적 s1 누설 ($B1 < B2$)

공통 원리: 마스크 y·챌린지 c·응답 z의 무결성이 깨지면 공개값만으로 s1이 역산된다. IRV는 각 지점을 재계산·재검사로 되짚어 잔차로 만든다

THE MAP

알고리즘 2 ↔ 구현 매핑 (L1~L10)

라인	검사 (뜻)	막는 오류	잔차 만드는 법
L1	$\delta \leftarrow 0$ (초기화)	—	<code>cm_delta=0</code>
L2	M1 비밀키 체크섬	영속 키변조	<code>FNV1a(s1) ≠ 기준 → $\delta =1$</code>
L3	M2 시드 정상성	SEED	<code>seed</code> 쏠0/쏠동일 → $\delta =1$
L4	M3 (y,b) 재유도	SIGNBIT	<code>(y',b') ≠ (y,b) → $\delta =1$</code>
L5	M4 응답 재계산	T1 / T2	<code>z ≠ y+(-1)^b c·s1' → $\delta =1$</code>
L6	M5 노름 B1+B0 재검사	RB	<code> z >B1 또는 bimodal 위반 → $\delta =1$</code>
L7	M6 표준 검증	LSB / UNPACK	<code>verify 실패 (main 분기)</code>
L8	마스크 m 생성	—	<code>factor = $\delta=0?0x00:0xFF$</code>
L9	감염 XOR	—	<code>sig ^= ks & factor</code>
L10	return σ	—	—

L2~L6은 무분기로 δ 에 누적(2차 오류에 강함). L7(표준검증)만 main에서 분기 — 상류 LSB/UNPACK 보강용.

SIDE BY SIDE

원본 HAETAE vs IRV — 라인별 비교

단계	원본 (sign.c)	IRV 적용 (haetae_sign_cm.c)
① 응답 z 계산	<code>cs = c·s1; z = y+(-1)^b·cs</code>	(동일)
② 거부 판정	<code>reject = z >B1 ∨ bimodal B0 if (reject) goto reject</code>	(동일)
③ 무결성 검사	— 없음 —	δ ← (z ≐ y+(-1)^b·c·s1' 재계산) (z ≤B1·B0 재검사) (시드·(y,b) 재유도)
④ 감염	— 없음 —	sig ^= ks & factor (δ≠0 이면 서명 난수화)
⑤ 패킹·반환	<code>pack_sig; return</code>	<code>pack_sig; infect_sig(δ); return</code>

핵심 연산(①②⑤)은 원본과 동일 — IRV는 ③검사·④감염만 삽입. 무결함이면 서명 바이트동일, 비용 1.14x.

L1 · L2

L2) M1 — 비밀키 체크섬 (막는 오류: 영속 키변조)

L1: $\delta \leftarrow 0$ 으로 시작(누산기). L2: 서명 시작 시 $s1$ 의 FNV-1a 체크섬을 기준값과 비교.

	$ck = \text{FNV1a}(s1)$	기준 golden	잔차
무결함	0x8F3AC21D	0x8F3AC21D	0
키 1비트 변조	0x2B71E04A	0x8F3AC21D	$\delta \neq 0$

```
uint32_t ck = ck_polyvecm(&s1);
if (!g_have_ck){ g_golden_ck = ck; g_have_ck = 1; }
if (ck != g_golden_ck) cm_delta |= 1;    // 다르면 잔차
```

FNV-1a: $h=2166136261$; 워드마다 $h^+=word$; $h*=16777619$ — 1비트만 바뀌어도 해시 전체가 달라진다.

L3) M2 시드 정상성 · L4) M3 (y,b) 재유도

L3 · M2 — 막는 오류: SEED

무결함 seed=3A 91 7C... → allz=0 → 잔차 0

SEED 스킵 seed=00 00 00... → allz=1 → $\delta \neq 1$

```
for(t..) { if(seed[t]) allz=0;
            if(seed[t]!=seed[0]) alls=0; }
if (allz | alls) cm_delta |= 1;
```

L4 · M3 — 막는 오류: SIGNBIT

무결함: 재유도 bb=0 = 방출 b=0 → 잔차 0

b 반전: 재유도 bb=0 ≠ 방출 b=1 → $\delta \neq 1$

```
sample_hyperball(&y1b, &y2b, &bb, seed, cb);
if (bb!=b || y1b!=y1 || y2b!=y2)
    cm_delta |= 1;
```

SIGNBIT은 부호만 뒤집혀 서명이 여전히 유효 → 검증·응답재계산으로 못 잡음 → 같은 시드로 재샘플해 (y,b) 대조로 포착.

L5 ★

L5) M4 — 응답 재계산 (막는 오류: T1 / T2)

$c \cdot s1$ 을 다시 곱해 $z1b = y1 + (-1)^b \cdot c \cdot s1$ 를 만들고 방출 $z1$ 과 계수별 비교. 오류는 출력의 y 를 지웠어도 $y1$ 은 메모리에 그대로!

	방출 $z1$	재계산 $z1b$	잔차
무결함	107	107	0
T2 (+y 스킵)	7	107	$\delta \neq 1$
T1 (c·s 스킵)	$\approx 100 (=y1)$	107	$\delta \neq 1$

```
for(ii=1..L) { pointwise(cs1b[ii],chat,s1[ii-1]); invntt(cs1b[ii]); }
cneg(cs1b,b); add(z1b, y1, cs1b); // z1b = y1+(-1)^b c·s1
if (z1[...] != z1b[..]) cm_delta |= 1; // 계수별 비교
```

예: 한 계수 $y1=100$, $c \cdot s1=7 \rightarrow$ 정상 $z=107$. T2는 방출 7(y 제거) vs 재계산 107 $\rightarrow$ 불일치 $\rightarrow$ 잔차.

L6

L6) M5 — 노름 B1+B0 재검사 (막는 오류: RB)

방출 z 의 노름을 서명경계 B1(과 bimodal B0)로 다시 검사. 검증경계 B2가 아니라 서명경계 B1로 보는 것이 포인트 ($B1 < B2$).

	$\ z\ $	$r1 = \ z\ > B1 ?$	잔차
무결함	8500	0 ($\leq B1$)	0
RB (거부 스킵)	10000	1 ($> B1$)	$\delta \neq 1$

```
r1 = (B1SQ*LN*LN - sqnorm2(z1,z2)) >> 63;    //  $\|z\| > B1$  이면 1
r2 = (sqnorm2(2z-y) - B0SQ*LN*LN) >> 63;    r2 &= (b>>1)&1;
if ((r1 | r2) & 1) cm_delta |= 1;
```

$B1=9839$ (서명경계) < $B2=12777$ (검증경계). RB의 $z(10000)$ 는 B2 검증은 통과하지만 B1 재검사에서 잡힌다.

VERIFY

L7) M6 — 표준 검증 (막는 오류: LSB / UNPACK)

방출 서명을 검증식으로 재구성. 자기일관적이지만 공개키와 안 맞는 서명(LSB→다른 c, UNPACK→잘못된 A)을 잡는다.

```
int bad = crypto_sign_verify(g_sig, g_slen, g_msg, 16, PRE, 0, pk);  
if (bad) memset(g_sig, 0xFF, CRYPTO_BYTES); // 무효화
```

스택이 커서 서명함수 밖(main)에서 수행. 알고리즘 2에서는 $\delta \mid= \text{Chk_M6}$ 로 이상화하지만, 실제 구현은 main의 분기다.

핵심 응답검사(L2~L6)는 무분기로 δ 에 누적 → 2차 오류에 강함. L7은 상류(LSB/UNPACK) 보강용이라 분기여도 무방.

THE INFECTION

L8) 마스크 생성 · L9) 감염 — δ 로 서명을 무효화

```
factor = 0u - (uint8_t)((delta | (0u-delta)) >> 63); // L8
sig[i] ^= ks[i] & factor; // L9 (ks = SHAKE256(비밀시드 ||  $\delta$  ||  $\mu$ ))
```

$\delta = 0$

```
factor = 0x00
sig ^= ks & 0x00 = sig
→ 서명 그대로 (오탐 0)
```

$\delta \neq 0$

```
factor = 0xFF
sig ^= ks & 0xFF = sig ^ ks
→ 서명 전체 난수화 (무효)
```

if 없이 “ δ 가 0인가?”를 전-0/전-1 마스크로. μ (메시지)를 섞어 마스크 신선도 확보. 버릴 분기가 없어 2차 오류가 성립 불가.

2차 오류로 잔차·감염 연산 자체를 건너뛰면? — 정직한 한계

질문: IRV의 잔차(δ)·감염 라인에 2차 오류를 걸면 그 계산도 건너뛸 수 있지 않나?

모델 O — 탐지 “분기” 스킵: IRV 핵심엔 if 분기가 없다($\text{sig} \neq \text{ks}$ & factor 는 무조건 실행) → 건너뛸 분기가 없어 생존. double-leeha는 분기 하나가 방어 전체라 이 한 방에 뚫린다.

모델 X — 잔차·감염 “연산 자체” 표적 (지적하신 시나리오): $\delta \neq 1$ 스킵 / δ 를 0으로 강제 / factor 계산 스킵 / infect_sig의 XOR 루프 스킵 → 누설 서명이 감염되지 않고 방출 → **IRV도 뚫린다**. 단 이건 “분기 스킵”이 아니라 특정 데이터 연산을 정밀 타격하는 다른 종류의 2차 오류다.

그래도 IRV가 나은 점 · 논문 입장: IRV는 가장 쉬운 단일 표적(중단 분기)을 제거해 “불리언 하나 뒤집기”를 “특정 감염 연산 정밀 타격”으로 격상시킨다. 2차 내성 주장은 분기-스킵 모델에 한정하며, 감염을 데이터흐름에 직조해 이 잔존 위협까지 건디는 강건 구현은 향후 과제(\$5.6).

요약

- 잔차 δ = 표적별 검사 6개(키체크섬·시드·(y,b)재유도·응답재계산·노름B1+B0·표준검증)를 한 번씩 수행해 OR로 누적.
- 원본 Sign 전체 재실행이 아니다 — 응답연산 1회 재계산 + 가벼운 재검사뿐. 그래서 1.14× (double 2.0×의 절반).
- 하나라도 잔차 $\neq 0 \rightarrow \delta \neq 0 \rightarrow \text{factor} = 0xFF \rightarrow$ 서명 전체 난수화. $\delta = 0$ (모두 통과)만 서명 보존(오탐 0).
- 마스크는 분기 없는 한 줄(factor). 버릴 if가 없으니 그 분기를 건너뛰는 2차 오류가 성립하지 않는다.

```
 $\delta$  = Chk_M1 | Chk_M2 | Chk_M3 | Chk_M4 | Chk_M5 | Chk_M6      →      sig ^= PRF(sk_seed ||  $\delta$  ||  $\mu$ ) & factor
```